Verify, Diagnose, Adapt: Agent-Validated Prerequisite Graphs for Student-Adaptive Learning Paths

Shreeya Kiran Khairnar
Dept. of Computer Science & Engg.
M S Ramaiah Institute of Technology
Bengaluru, India
shreeyakiran04@gmail.com

Vaidehi Kumawat
Dept. of Computer Science & Engg.
M S Ramaiah Institute of Technology
Bengaluru, India
vaidehikumawat2020@gmail.com

Sneha Debnath
Dept. of Computer Science & Engg.
M S Ramaiah Institute of Technology
Bengaluru, India
snehadebnath2608@gmail.com

Shrestha Sunil
Dept. of Computer Science & Engg.
M S Ramaiah Institute of Technology
Bengaluru, India
shrestha.sunil051@gmail.com

Sangeetha J
Dept. of Computer Science & Engg.
M S Ramaiah Institute of Technology
Bengaluru, India

***Abstract*—Students often struggle with an advanced topic because they missed a concept it depends on, yet course material rarely states which concepts depend on which, or which gap a particular student has. Large language models (LLMs) can propose prerequisite relations cheaply, but they also propose relations that sound right without being needed. We present an end-to-end system that turns course files into a personalized, explainable learning path. An LLM proposes prerequisite edges, and an independent validation agent checks each one against course text with verified quotes, a judgment from a second model, and exact graph rules, then accepts it, rejects it, or sends it for human review. An adaptive quiz over the validated graph estimates what a student knows, and the student receives a path with cited sources. Early runs exposed three findings. A counterfactual "necessity test" was unstable on three example edges, so it does not influence decisions. On a real textbook, 12 of the 13 sampled concept pairs that received a link changed when the two concepts were swapped in the prompt, so course runs now ask both orders. On 500 simulated students over a five-concept graph, adaptive diagnosis used 14.4% fewer questions than asking every concept at similar accuracy (0.866 vs. 0.870), but only when knowledge followed the graph. A walkthrough on an AVL-tree slide deck shows how the agent keeps the graph acyclic and sends links proposed in both directions to a teacher. This is work in progress: the main evaluation of edge validation on public benchmarks and full courses is pending, and all numbers are preliminary.**

***Index Terms*—prerequisite relation discovery, educational knowledge graph, large language models, LLM validation, adaptive diagnosis, personalized learning paths**

I. Introduction

Learning is cumulative. A student who says "I don't understand AVL trees" may have no trouble with balancing at all; they may never have understood the ordering rule of binary search trees (BSTs). Telling them to revise AVL trees does not find that gap. A useful answer depends on two things: the dependency structure of the course, and where in that structure this student's knowledge stops.

An instructor could draw a prerequisite map by hand, but this is slow for a course with hundreds of concepts, since the number of candidate concept pairs grows quadratically. LLMs can propose such maps at low cost [1], but they also generate plausible errors [2]. In a graph, one wrong edge adds an unnecessary branch to every learning path that passes through it, and a cycle destroys the ordering needed to produce a path at all.

We ask two linked questions. Can LLM-proposed prerequisite edges be checked, so that only supported edges enter the course graph? And can one validated course graph give each student a different path, based on what that student already knows?

We do not claim that prerequisite discovery is new. Our contributions are:

1. An independent validation agent that combines course-text evidence with verified quotes, a semantic judgment from a different model, and graph-consistency rules, with three outcomes: accept, reject, or human review.
2. A measurement of how LLM proposals depend on question order, and a discovery rule that asks both orders and keeps the disagreement visible.
3. A study of a counterfactual necessity test, in which an LLM simulates a learner who lacks a concept. We report that it was unstable in our runs.
4. Graph-guided adaptive diagnosis, which uses the validated graph to choose the next quiz question and to infer knowledge of related concepts.
5. Root-cause tracing, which reads the option a student chose and ranks which concept in the prerequisite chain most likely caused the error.
6. Personalized, explainable paths and impact-ranked human review: one course graph, a different path per student, with uncertain edges ranked by how many student paths they would change.

The system is implemented end to end, including a website that takes uploaded course files.

II. Related Work

A. Prerequisite Relation Learning

Early work used Wikipedia links and crowdsourced judgments [3]. RefD measures asymmetry in links between related pages [4]. Others learned from MOOC features [5], university course dependencies [6] and active learning [7]. PREREQ [8] and LectureBank [9] moved toward learned representations. More recently, PCPL applies prompt learning [10], LCPRE uses learning-path information [11], and GKROM optimizes relations globally [12]. Most of this work measures pairwise accuracy on curated benchmarks. We treat discovery as an input and focus on what happens after an LLM proposes an edge.

B. Educational Knowledge Graphs and Paths

ACE combines machine learning with expert input to reduce the effort of building prerequisite graphs [13]. InstructKG builds graphs from lecture material and reports that LLM extraction without instructor-grounded signals yields inaccurate pedagogical graphs [14], which matches what we observed. Prerequisite information has also been used to recommend learning objects [15]. Our paths differ per student, and each step keeps its evidence.

C. Student Modeling

Knowledge Space Theory models feasible knowledge states under a prerequisite relation, so a few questions can place a learner in a state [16]. Our diagnosis uses the same idea, but the relation is proposed by an LLM and checked automatically instead of being elicited from experts. Knowledge tracing estimates mastery over time from many interactions [17], [18]. A new course has no such logs, whereas our approach starts from course material alone.

D. Verification of LLM Outputs

Common countermeasures include grounding in retrieved text [19], sampling and majority voting [20], and using one model to judge another [21]. We apply these ideas to a structured output, a graph edge, where some constraints such as acyclicity can be checked exactly.

III. Problem Formulation

A course is a set of documents D split into chunks K, each keeping its source document and page. Extraction gives a set of concepts C. A concept A is a *hard* prerequisite of B if B cannot be understood without A, and a *soft* prerequisite if A clearly helps but is not required. The course graph $G = (C, E)$ is a directed acyclic graph (DAG) of validated edges, shared by all students. Fig. 1 shows the system.

What differs per student s is a knowledge state $m_s \in [0, 1]^{|C|}$, a mastery value for each concept. A concept counts as mastered when $m_{s,c} > \tau = 0.8$. For a target t, the personalized path is the target and its ancestors minus the mastered concepts, in topological order:

$$\pi(s,t) = \mathrm{TopSort}\left(\left(\{t\} \cup \mathrm{Anc}_G(t)\right) \setminus \{c : m_{s,c} > \tau\}\right) \quad (1)$$

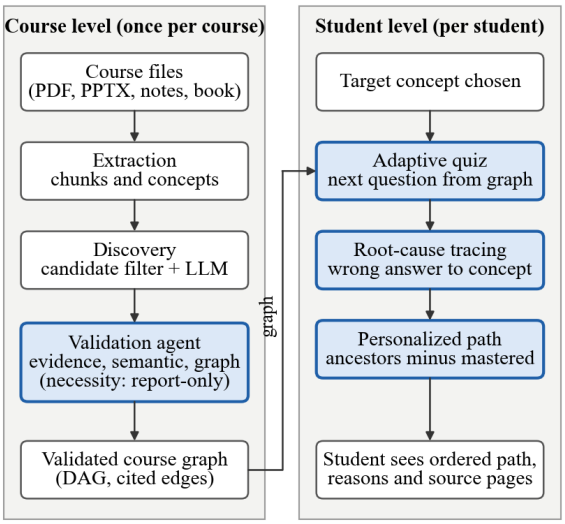


Fig. 1. System overview. The course level runs once per course and the student level runs per student. Every stage exchanges schema-validated JSON files.

IV. Proposed System

A. Extraction

PDF, PPTX, Markdown, TXT and DOCX files are split into chunks of at most 500 words that keep their source. An LLM lists the concepts in each chunk, and near-duplicate names are merged by embedding similarity (all-MiniLM-L6-v2 [22]), with aliases kept. An evidence filter then keeps a concept only if it appears in a section heading or has enough mentions or source chunks outside exercise and summary sections. Extraction stops with an error instead of silently falling back if the LLM or the embedding model is unavailable, and the method used is recorded in the output.

B. Dependency Discovery

Sending every concept pair to an LLM is too expensive, so a pair is kept only if there is a signal: the concepts share a chunk, one is named in the other's definition or source text, or their embeddings are similar. The proposer (qwen2.5:7b-instruct) receives up to four excerpts and answers one neutral question: *A* before *B*, *B* before *A*, or neither, with a strength and a confidence. Which concept is called *A* comes from a hash of the pair, so position carries no information about the true direction (prompt version v3.1).

Because the answers depend on that position (Section VI-B), course runs ask every pair in both orders under a rule written down before any full run. The same direction twice gives a *stable* edge. A link in one order and "neither" in the other gives an *unstable* edge. Opposite directions give a *reversed* pair, and both edges go to the validator. No link in either order gives no edge. Benchmark runs keep the standard protocol and ask each pair once, in the dataset's own orientation, so they cannot show this effect.

A newer prompt, discover_v4, removes the *A*/*B* labels altogether. It asks one yes/no question per direction, "does a student need *X* to understand *Y*?", so each pair costs two calls with the same excerpts. One yes gives one proposed edge; yes

in both directions proposes both edges, which go to review and are never resolved automatically. Replies that do not name the pair count as no, and both raw answers are kept. It is used for the walkthrough in Section VI-I and is not yet compared with v3.1.

C. Validation Agent

Every proposed edge is treated as a claim. The agent uses a different model (llama3.1:8b) and its own prompts, because a model that finds an edge plausible once will tend to do so again. All calls use a fixed seed and are cached, with the model, prompt version and temperature recorded.

Graph check (no LLM). Each edge is labelled self-loop, duplicate, contradiction (both directions proposed), cycle, redundant (implied by a longer path) or ok.

Evidence check. The agent gathers the chunks discovery cited plus the top three retrieved by embedding similarity. The model grades whether the excerpts support learning *A* before *B* as strong, medium, weak or none, and it must quote the text. A quote is kept only if it appears verbatim in the cited chunk, and a strong or medium grade with no verifiable quote is capped at weak. Every displayed quote is therefore traceable to a document and page.

Semantic check. The model grades the edge from concept names and descriptions only, so it does not share the evidence check's input. Used alone, it is also our single-judge baseline.

Necessity test. A question writer creates questions on *B* without naming *A*. A simulated learner answers them with *A* forbidden. A control run uses the same prompt with an unrelated concept forbidden instead, which absorbs the penalty of being told "you never learned X". A separate grader, blind to the run, scores both. The necessity gap is the control score minus the without-*A* score. Because of the instability reported in Section VI-D, this test is report-only in the current agent: it has weight 0 and never changes a decision.

Decision rule. Self-loops and duplicates are rejected, and redundant edges are rejected with the reason "implied by ...". Contradictory or bidirectional edges go to human review, and a cycle is never accepted. Otherwise grades map to points (1, 0.67, 0.33, 0), and the score is the mean over active checks. An edge is accepted if the score is at least 0.6 and no active check says none. It is rejected if the score is below 0.35, or if evidence and semantic are both weak or none. Otherwise it goes to review. These values are a first choice, not yet calibrated.

Which edges count as structure. Under order_rule_v2, edges proposed in both directions and edges rejected on their scores do not count in other edges' cycle and redundancy checks, since neither is a claim the graph should act on. Only edges labelled ok can be accepted, so the accepted graph is always a DAG. This rule was chosen after seeing the demo deck of Section VI-I, not on gold labels.

D. Impact-Ranked Human Review

For each edge e sent to review, we compare every simulated student's path with and without the edge. Its impact is the share of paths that change,

$$\text{Impact}(e) = \frac{1}{|S|}\sum_{s\in S} \text{I}[\pi^{+e}(s,t) \neq \pi^{-e}(s,t)] \quad (2)$$

and the queue is sorted by impact. Edges that would close a cycle are flagged as blocking and come first. On the website, a teacher accepts or rejects an edge, and an edge that would close a cycle is never added.

E. Adaptive Diagnosis and Personalized Path

Each concept starts at mastery 0.5. The next question targets a concept near the middle of the uncertain part of the target's prerequisite chain (mastery between 0.2 and 0.8). A correct answer sets that concept to 0.9 and its ancestors to at least 0.85. A wrong answer sets it to 0.1 and keeps its prerequisites in question. The latest direct answer on a concept always overrides inference. A quiz stops at 10 questions per target; if that limit is reached, concepts that were not asked stay on the path.

Quiz items are generated by an LLM from the course chunks, and each must carry a word-for-word quote from a cited chunk. Code checks then drop an item if the quote is absent, if the question names a concept its source does not mention, or if under half of the answer's content words appear in the quote. A second model, not told the answer, judges each option from the excerpt and can veto the item. A topic left with fewer than two items gets one retry that states why its drafts failed.

Items are written at three levels (easy, medium and hard). On the website, a student first says how well they know a topic, which only sets the starting level, and then climbs or drops a level per answer. A correct hard answer marks the topic known; a correct medium answer with a wrong hard one gives quick revision; a correct easy answer with a wrong medium one gives study; a wrong easy answer means start from the basics, and only topics that depend on it through hard edges are then held back. The simulations in Section VI-E use the mastery rule above, not the ladder.

F. Root-Cause Tracing

Given a wrong answer on concept *T*, the candidates are *T* and its ancestors over accepted edges. An LLM reads the question, the options, the chosen option and this chain, and ranks the candidates with a one-line explanation. Output is restricted to the chain. Two baselines also rank the chain: blame the target, and blame the nearest prerequisite (lowest mastery, then hard before soft, then concept ID).

V. Experimental Setup

A. Datasets

Our courses use openly licensed material so the work can be reproduced (Table I): ds101 uses Open Data Structures [23] and OpenDSA modules [24] for AVL trees, general trees and

recursion, and ml101 uses Dive into Deep Learning [25] and Introduction to Modern Statistics [26] for logistic regression.

TABLE I
OUR COURSE CORPORA

Course	Files	Chunks	Concepts	Gold rows
ds101	13	149	259 (85 after filter)	9 (pilot)
ml101	10	78	pending	4 (pilot)
AVL demo	1	33	26 (34 before filter)	none

Gold rows exclude 7 tied pairs awaiting adjudication. ds101 also has 120 set 2 pairs from one annotator (Section V-C). The AVL demo is a slide deck written by the authors for the walkthrough.

Public benchmarks are AL-CPL [7] and the University Course dataset [6] (Table II). Neither provides course text, so on these the agent uses only the semantic and graph checks. Their positives include many pairs implied by other positives, so we score the transitive closure of our predictions against them; our own gold labels are direct edges only. University Course contains 99 votes with a label its documentation does not define, and we drop them. LectureBank [9] is excluded because its repository and paper state no licence for its annotations.

TABLE II
PUBLIC BENCHMARK STATISTICS

Graph	Concepts	Pos.	Neg.	Implied
AL-CPL data mining	120	292	534	180
AL-CPL geometry	89	524	1157	414
AL-CPL physics	153	487	1475	308
AL-CPL precalculus	224	699	1361	454
University Course	521	1007	1821	555

Implied: positives already implied by a path of other positives.

B. Research Questions and Baselines

- **RQ1:** Does the agent improve the precision of LLM-proposed edges, and at what cost to recall? Compared with a baseline prompt, discovery alone, a single judge, and a self-consistency judge (three samples at temperature 0.7), plus ablations removing each check. We also report the human-review rate and LLM calls per edge.
- **RQ2:** Does adaptive diagnosis need fewer questions than asking every concept, at similar accuracy? Also compared with random order at the same budget.
- **RQ3:** Does LLM tracing beat the two baselines? Measured by top-1 and top-3 accuracy on a 40-item test set, labelled by a different author from the one who wrote the prompt.
- **RQ4:** Are personalized paths shorter than generic paths while still covering real gaps? Measured by path length, gap coverage and order violations.

C. Annotation and Sampling

Annotators label ordered pairs as hard, soft or none under written guidelines, each in their own file. We report Cohen's κ [27] raw and after a mechanical shortcut rule, which turns a pair into none when that annotator's own labels already give a path through another concept. Thresholds will be calibrated on a development split that is excluded from all reported scores. On benchmarks, each graph is scored on a seeded, label-stratified sample of 200 pairs, posed in the dataset's own orientation, with a nested 100-pair subset for the self-consistency judge.

A second ds101 set of 120 pairs among the filtered concepts was labelled by one author without AI help or access to model output: 16 hard, 42 soft and 62 none, split before any scoring into 36 development and 84 test pairs. With a single annotator it has no κ, so it is a gold set with known limits rather than an agreed one.

D. Simulated Students and Environment

Students are sampled from a learning frontier so that knowledge follows the graph, with slip (0.05–0.12) and guess (0.10–0.20) noise. A second condition ignores the graph, with each concept mastered independently at $p = 0.5$. Models run locally through Ollama, using only free tools, on an Apple M2 laptop with 8 GB of memory. Recent discovery calls averaged 8.4 s with qwen; evidence and semantic calls took medians of 26 s and 8.5 s with llama.

VI. PRELIMINARY RESULTS

The scoring harness, baselines, ablation flags and benchmark sampler for RQ1 are implemented, but no RQ1 run is complete, so we report no precision, recall or F1. RQ3 has only a development set. The findings below are preliminary, and none is a final evaluation; Section VI-I is an illustration only.

A. Discovery on a Real Textbook

All 259 extracted ds101 concepts produced 6,088 candidate pairs out of 33,411, too many for our hardware. Many concepts were too fine-grained, such as "add(i, x)". We therefore tried a subset of 30 core concepts, which gave 233 candidates out of 435 pairs and 82 proposed edges.

Those 82 edges were poor. Our key chain, binary tree → BST → AVL tree, was missing, while 19 edges pointed into "heap" and 17 into "general tree", and every edge had confidence 0.9. Ollama had truncated the prompt in 200 of 237 calls, which removed the question: only 36 of 233 matched replies named the pair they were asked about. Direction also came from the order of concepts in the files, and the reverse direction was never asked. A run with a GPU out-of-memory error during embedding also kept all 435 pairs, so such runs are now discarded.

Prompt v3 asked one neutral question with limited excerpts and the question last. On the same candidates it proposed 64 edges, with "binary tree" now the most common target, but its answers were lopsided: 59 "*B* before *A*" against 5 "*A* before *B*", while *A* was always the lower concept ID. General tree → binary tree and binary tree → BST were still not proposed.

B. Sensitivity to Question Order

To test that imbalance, we asked 35 pairs from the 85 filtered ds101 concepts in both orders with v3.1: the 5 positive gold pairs and a seeded sample of 30 other candidates. Table III gives the outcome. Of the 13 pairs where either order proposed a link, 12 changed when *A* and *B* were swapped. For

BST and AVL tree the direction itself reversed, so the model followed the position rather than the concepts. Across all 70 asks the answers were 14 "*B* before *A*", 4 "*A* before *B*" and 52 neither; these are paired questions on one course, so we read the preference as a pattern, not a rate.

TABLE III
RESULT WHEN THE TWO CONCEPTS ARE SWAPPED

Result across both orders	Gold	Sample	All
Same answer (22 of 23: neither twice)	2	21	23
Direction reversed	2	2	4
One order neither, the other a link	1	7	8

Two gold IDs were mapped to the filtered concepts for this test: trees to general tree, and graphs to graph.

The hash stops position from lining up with the true direction, but the answers stay unstable, which motivates the both-order rule. Its reasoning comes from the 30 non-gold pairs, but the 5 gold pairs were also visible when it was chosen.

To check that the stages work together, we ran the validator on 28 of the 64 v3 edges, chosen by a written rule that favored tree-related edges and edges that looked likely to be wrong. It accepted 6, rejected 16 and sent 6 to review, with no prompt truncation. Because the selection was deliberate, these counts are not a performance measure.

C. Checks on Example Edges

We ran the evidence check and semantic prompts once on six example edges (llama3.1:8b, temperature 0, seed 42). These edges overlap the gold labels, so this shows behavior and is not an evaluation. Both the evidence check and semantic v2 graded the unrelated edge, graphs → AVL tree, as none. Moving from semantic v1 to v2 raised binary tree → BST from weak to strong, as intended, but also over-graded the soft edge recursion → binary tree as strong. The evidence check graded binary tree → BST only weak.

D. Necessity Test Stability

We expected the gap to follow binary tree → BST > recursion → binary tree > graphs → AVL ≈ 0. No setting reproduced this order (Fig. 2). Open questions at $n = 3$ gave every edge the smallest possible step, so they could not rank. Multiple-choice runs mostly gave identical control and without-*A* scores, including a zero gap for the hard edge. At $n = 5$ with sampled questions, the unrelated edge scored above the hard one. One question there named *A*, which we now prevent. At temperature 0, seeds repeat the same output, so an SD of 0 reflects repetition, not stability.

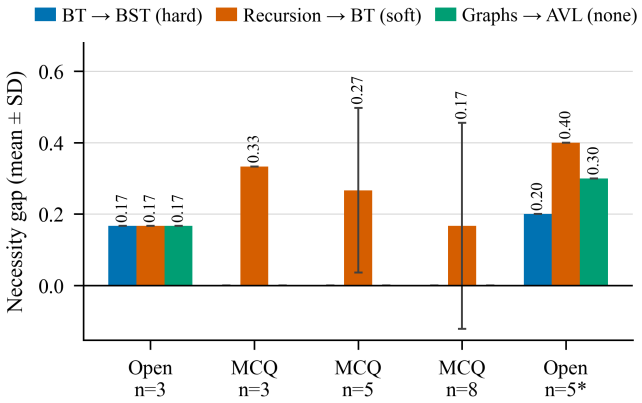


Fig. 2. Necessity gap by setting and edge. Error bars show SD across three seeds. *One seed, questions written at temperature 0.7.

A plausible explanation is that a small LLM cannot reliably simulate not knowing a concept, since it answers partly from its own knowledge. We have not tested this directly. Three edges cannot support a general conclusion, so RQ1 includes a setting that gives the test weight, to measure its effect on real data.

E. RQ2 and RQ4: Simulated Students

We simulated 500 students per condition on a five-concept example graph, target AVL tree (Fig. 3). When knowledge follows the graph, adaptive diagnosis saved 14.4% of questions at nearly the accuracy of asking everything, and beat random order at the same budget. When knowledge ignores the graph, it saved 27.2% of questions but lost about 10 accuracy points, so its advantage depends on the graph being a good model of the student.

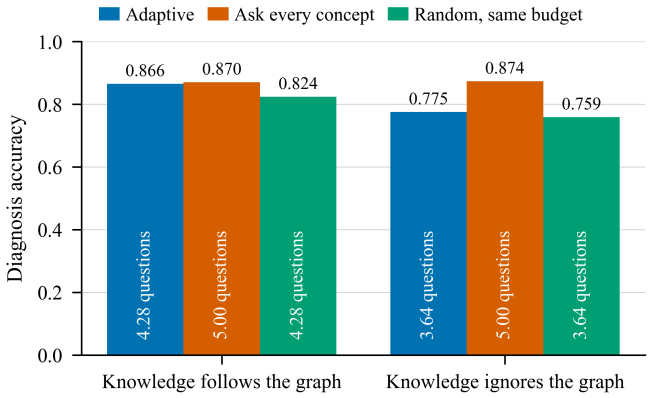


Fig. 3. Diagnosis accuracy by strategy and student condition (RQ2). Each bar states the average questions asked.

Personalized paths averaged 2.35 concepts (follow graph) and 1.77 (ignore graph) against a generic path of 5, a reduction of 52.9% and 64.6% (RQ4). They do not cover every gap, because diagnosis makes errors: gap coverage was 0.81 and 0.59, so the shorter path comes with far lower coverage. Order violations were 0, but that follows from topological sorting of the same graph, so it says little by itself.

F. Quiz Item Checks

We replayed 32 real draft items for 8 ds101 topics through the checks in Section IV-E. Code checks stopped 20 of them, 12 reached the second model, and 8 were kept. An author's reading was mixed: the checks removed clear errors, such as questions naming a red-black tree with a quote about AVL trees, but also dropped items that read well, and some weak items survived. These items shaped the checks, so this gives no accuracy figure, but it suggests automatic checks need a teacher review step.

G. Root-Cause Tracing (Development Set)

On a 10-item development set with no mastery information, the LLM tracer tied the blame-the-target baseline (top-1 0.5, top-3 0.8), and blame-the-nearest reached top-1 0.4, top-3 0.8. Ten items cannot separate these methods. The tracer is now frozen, and it will be evaluated once on the 40-item test set, which shares no quiz items with the development set.

H. Inter-Annotator Agreement (Pilot)

Two authors labelled 20 pilot pairs, chosen before the real concept list existed (Fig. 4). The rule changed four labels of one annotator and none of the other's, because the second annotator's labels had already had the rule applied during labelling, so raw κ partly reflects that difference. Seven pairs are ties awaiting adjudication.

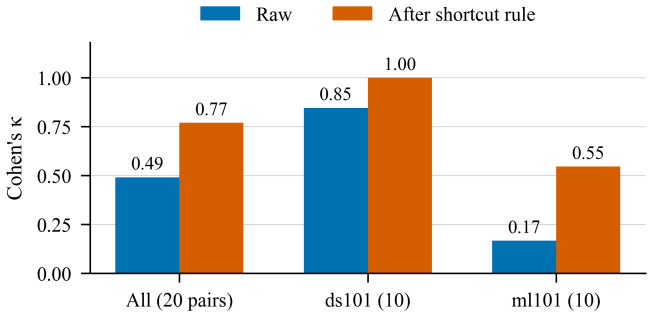


Fig. 4. Cohen's κ between two annotators, raw and after the shortcut rule, before any adjudication.

I. Walkthrough: An AVL Trees Slide Deck (Not an Evaluation)

To show the whole system on one input, we ran it on a 33-slide deck on AVL trees that the authors wrote for the demo, with discover_v4 and order_rule_v2, on a free Colab T4 GPU (repository commit e1f909c). Both rules were chosen after seeing this deck's results under the earlier rules, so we give no counts from it as evidence. Fig. 5 is the accepted graph as the website's topic map draws it for a student: solid arrows are hard prerequisites and dashed arrows soft ones.

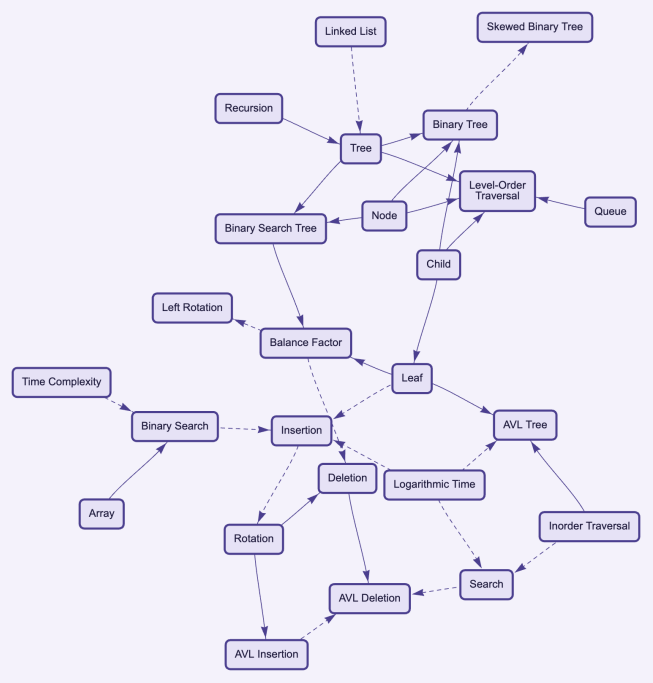


Fig. 5. Accepted prerequisite graph of the AVL demo deck, drawn by the website's topic map code, with larger labels for print (solid: hard, dashed: soft). An arrow points from the topic to learn first.

Fig. 6 follows the intended chain tree → binary tree → BST → rotation → AVL tree through the validator's decisions.

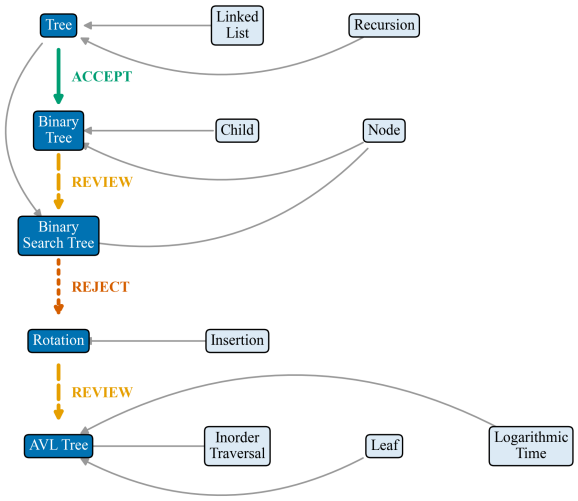


Fig. 6. Validator decisions on the AVL demo chain (coloured), with the accepted edges into each chain topic (grey). Two links were proposed in both directions and await a teacher; one was rejected as redundant.

Tree → binary tree was accepted with a verbatim quote from slide 12 ("A Binary Tree is a Tree in which every node has at most two children"). For binary tree → BST the model answered yes in both directions, so both edges went to review. The checks already separate them: binary tree → BST scored strong on evidence and semantics, and BST → binary tree scored weak and none. This supports testing the pre-declared variant that lets the validator pick a direction. Rotation →

AVL tree went to review for the same reason, but there both directions scored strong, so a teacher is needed.

The deck's quiz uses quiz_v3 items at three levels with the ladder of Section IV-E. Reading the kept items, we still found wrong ones that had passed every check, so teacher approval of items is planned before any use with real students.

BST → rotation exposed a flaw. It was rejected as implied by BST → insertion → rotation, but BST → insertion had itself been rejected as redundant. A rejected edge should not make another edge redundant, and this will be fixed before any reported run. Until a teacher resolves the review items, the accepted graph does not connect binary tree or BST to AVL tree, so a student's AVL path cannot include them. The walkthrough therefore shows the intended safety property, an acyclic graph with doubtful edges held back, and also why the human review step is part of the system.

VII. Limitations

- **Example graph.** RQ2 and RQ4 use five concepts, not a real course.
- **Simulated students share the graph they evaluate.** The first condition shows performance given a correct graph, not that real students behave this way.
- **One right answer can be a guess.** Diagnosis usually asks one question per concept, so a lucky guess can mark a concept as known.
- **Small pilot gold.** Twenty pairs by two student annotators cannot support strong conclusions, and only 5 positive edges back ds101.
- **Benchmarks test only part of the agent.** They ask one fixed order, so they cannot reveal order sensitivity, and they use a different discovery prompt (v2) from our courses (v3.1). Both come from the same group.
- **One prompt change on pilot edges.** Semantic prompt v2 was changed once after we saw example edges that overlap the pilot gold, so those pairs must be excluded from final scores.
- **Discovery is not yet good enough.** The both-order rule exposes order sensitivity rather than removing it and has not been run at scale, so its effect on edge quality is unknown. The concept filter lost none of the 4 gold concepts with an exact ID match, but a 6-concept list is too small to judge it, and it is untested on ml101.
- **Rules chosen on the demo deck.** discover_v4 and order_rule_v2 were chosen after seeing the authors' own AVL deck, so that deck cannot be used to measure them.
- **Hardware.** Two models do not fit together in 8 GB of memory, which limited how much we could run.

VIII. Future Work

The immediate work is to complete the evaluation: run both-order discovery and validation on the filtered courses, run RQ1 on the benchmarks and full courses, label a larger gold set of real concepts, calibrate and freeze thresholds, then run RQ3 once and repeat RQ2 and RQ4 on real graphs. A small pre/post study with real students is planned.

IX. Conclusion

We built an end-to-end system in which LLM-proposed prerequisites are treated as claims to check. Validated edges then support an adaptive quiz and a personal learning path. Our early findings are mixed and useful. Simulating a learner's missing knowledge was unstable, a real textbook exposed silent failures in discovery, proposed links depended heavily on question order, and adaptive diagnosis saves questions mainly when knowledge follows the graph. On an AVL deck, the agent kept the graph acyclic but held key links for review, which a teacher must resolve. Whether the agent improves edge quality remains to be measured, and that is the next step.

Acknowledgment

The authors thank the Department of Computer Science & Engineering, M S Ramaiah Institute of Technology. Parts of the software and of this manuscript's text were drafted with an AI coding assistant (Anthropic's Claude, through Claude Code), including the mechanical application of the shortcut rule to one annotator's answers. The authors directed this work and remain responsible for it.

References

[1] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3580–3599, 2024.

[2] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[3] P. P. Talukdar and W. W. Cohen, "Crowdsourced comprehension: Predicting prerequisite structure in Wikipedia," in *Proc. 7th Workshop Building Educ. Appl. Using NLP*, 2012, pp. 307–315.

[4] C. Liang, Z. Wu, W. Huang, and C. L. Giles, "Measuring prerequisite relations among concepts," in *Proc. EMNLP*, 2015, pp. 1668–1674.

[5] L. Pan, C. Li, J. Li, and J. Tang, "Prerequisite relation learning for concepts in MOOCs," in *Proc. ACL*, 2017, pp. 1447–1456.

[6] C. Liang, J. Ye, Z. Wu, B. Pursel, and C. L. Giles, "Recovering concept prerequisite relations from university course dependencies," in *Proc. AAAI*, 2017, pp. 4786–4791.

[7] C. Liang, J. Ye, S. Wang, B. Pursel, and C. L. Giles, "Investigating active learning for concept prerequisite learning," in *Proc. AAAI (EAAI)*, 2018, pp. 7913–7919.

[8] S. Roy, M. Madhyastha, S. Lawrence, and V. Rajan, "Inferring concept prerequisite relations from online educational resources," in *Proc. AAAI*, vol. 33, 2019, pp. 9589–9594.

[9] I. Li, A. R. Fabbri, R. R. Tung, and D. R. Radev, "What should I learn first: Introducing LectureBank for NLP education and prerequisite chain learning," in *Proc. AAAI*, vol. 33, 2019, pp. 6674–6681.

[10] F. Du, W. Hu, C. Qin, and Z. Xiao, "A prompt-based approach for discovering prerequisite relations among concepts," in *Proc. ITEI 2023*, Zhengzhou, China, EAI, 2024, doi: 10.4108/eai.24-11-2023.2343577.

[11] J. Sun, Y. He, Y. Xu, J. Sun, and G. Sun, "A learning-path based supervised method for concept prerequisite relations extraction in educational data," in *Proc. CIKM*, 2024, pp. 2168–2177.

[12] M. Zhang, J. Wang, K. Xiao, S. Wang, Y. Zhang, H. Chen, and Z. Li, "Learning concept prerequisite relation via global knowledge relation optimization," in *Proc. AAAI*, vol. 39, no. 2, 2025, pp. 1638–1646.

[13] M. C. Aytekin and Y. Saygın, "ACE: AI-assisted construction of educational knowledge graphs with prerequisite relations," *J. Educ. Data Mining*, vol. 16, no. 2, pp. 85–114, 2024.

[14] A. AlRabah, P. Kargupta, J. Han, and A. Alawini, "Instructor-aligned knowledge graphs for personalized learning," arXiv:2602.17111, 2026.

[15] A. Chanaa and N. El Faddouli, "Prerequisites-based course recommendation: Recommending learning objects using concept

prerequisites and metadata matching," *Smart Learn. Environ.*, vol. 11, art. 16, 2024.

[16] J.-P. Doignon and J.-C. Falmagne, "Spaces for the assessment of knowledge," *Int. J. Man-Machine Studies*, vol. 23, no. 2, pp. 175–196, 1985.

[17] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapt. Interact.*, vol. 4, no. 4, pp. 253–278, 1994.

[18] C. Piech et al., "Deep knowledge tracing," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 505–513.

[19] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.

[20] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *Proc. ICLR*, 2023.

[21] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Adv. Neural Inf. Process. Syst. Datasets and Benchmarks Track*, 2023.

[22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[23] P. Morin, *Open Data Structures (in pseudocode)*, Edition 0.1Gβ. [Online]. Available: https://opendatastructures.org/

[24] OpenDSA Project, "OpenDSA modules," GitHub repository, commit f900a7a. [Online]. Available: https://github.com/OpenDSA/OpenDSA

[25] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, release 1.0.3. [Online]. Available: https://d2l.ai/

[26] M. Çetinkaya-Rundel and J. Hardin, *Introduction to Modern Statistics*, 2nd ed., OpenIntro, 2024. [Online]. Available: https://openintrostat.github.io/ims/

[27] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.